



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

COS 226 - Concurrent Systems

Practical 3 Specification

Test-and-Set Locks and Contention

Release Date: 31 August 2026

Submission Deadline: 6 September 2026

1 Overview

In this practical, you will implement and investigate a spin lock based on the *Test-and-Set* (TAS) operation.

A Test-and-Set lock repeatedly invokes an atomic `testAndSet()` operation while attempting to acquire the lock. Although this provides mutual exclusion, repeated atomic operations can introduce significant overhead when multiple threads compete for the same lock. In particular, repeated atomic operations can result in increased cache-coherence traffic between processor cores.

The practical consists of three tasks. You will first implement a basic Test-and-Set lock. You will then modify your implementation to reduce the unnecessary overhead associated with repeatedly invoking the atomic operation. Finally, you will experimentally compare the two implementations under increasing levels of contention.

The emphasis of this practical is on understanding how atomic operations behave under contention and how the number of atomic operations performed can affect the performance of a concurrent program.

2 Learning Outcomes

By completing this practical you should be able to:

- Implement a spin lock using an atomic Test-and-Set operation.
- Explain how a Test-and-Set lock provides mutual exclusion.
- Identify the performance overhead associated with repeatedly invoking an atomic operation under contention.
- Modify a Test-and-Set lock to reduce unnecessary atomic operations while preserving mutual exclusion.

- Explain the purpose of checking the lock state before performing an atomic Test-and-Set operation.
- Experimentally investigate the effect of increasing thread contention on lock performance.
- Analyse execution time and the number of atomic operations performed by different lock implementations.

3 Requirements

This practical is to be completed in groups of **three**. Group members are expected to contribute equally to the implementation, and all group members should be able to explain any part of the solution during the demonstration.

A skeleton implementation of the Test-and-Set lock will be provided. You must complete the required methods without modifying the provided `testAndSet()` operation.

The `testAndSet()` method represents an atomic Test-and-Set operation. You may not use Java's built-in locking mechanisms, including `synchronized` or `ReentrantLock`, to implement either lock outside of the provided `testAndSet()` method.

4 Task 1 – Implement a Test-and-Set Lock

Complete the `lock()` and `unlock()` methods to implement a correct Test-and-Set spin lock. Your implementation must:

- Use the provided `testAndSet()` operation to acquire the lock.
- Ensure that only one thread can enter the critical section at a time.
- Correctly release the lock when `unlock()` is called.
- Not modify the provided `testAndSet()` method.

Your implementation should be tested using multiple concurrent threads accessing a shared counter.

5 Task 2 – Optimising the Test-and-Set Lock

The Test-and-Set implementation from Task 1 repeatedly invokes the atomic `testAndSet()` operation while the lock is held.

Under high contention, many threads may repeatedly perform the atomic operation even though the lock is known to be unavailable. This can result in unnecessary atomic operations and increased cache-coherence traffic.

Modify your `lock()` implementation to reduce this unnecessary overhead.

The following restrictions apply:

- The provided `testAndSet()` method must not be modified.
- The implementation must continue to use `testAndSet()` to acquire the lock.
- Mutual exclusion must be preserved.
- Java's built-in locking mechanisms may not be used.
- The optimisation must reduce unnecessary invocations of `testAndSet()` when the lock is already held.

Hint: Consider whether you can check the current state of the lock before attempting an atomic `testAndSet()` operation.

You are encouraged to experiment with your implementation and may introduce additional optimisations, provided that they satisfy the requirements above and preserve mutual exclusion.

6 Task 3 – Contention Experiment

You will experimentally compare your original Test-and-Set implementation with your optimised implementation.

Run both implementations using increasing numbers of concurrent threads. At a minimum, test the following configurations:

- 2 threads
- 4 threads
- 8 threads
- 16 threads
- 32 threads

For each configuration, both implementations must perform the same workload.

Each configuration must be executed at least **five times**. Calculate the average execution time and average number of `testAndSet()` invocations.

Record the following measurements for each implementation:

- Average execution time.
- Average number of `testAndSet()` invocations.

Your results should be presented in a clear table.

You should expect some variation between individual runs. Differences in processor architecture, operating-system scheduling, JVM behaviour, and background system activity may affect the measured execution time.

7 Demonstration

Students are required to demonstrate their implementation to a tutor during their scheduled practical session.

During the demonstration, students must:

- Demonstrate that the Test-and-Set lock provides mutual exclusion.
- Demonstrate their optimised implementation.
- Demonstrate the contention experiment.
- Show the recorded average execution times and `testAndSet()` invocation counts.
- Explain the implementation and design decisions made in their solution.

Students should be prepared to explain their implementation and experimental results rather than simply demonstrate that the program produces the expected output.

Conceptual questions relating to Test-and-Set, TTAS, contention, atomicity, and cache coherence will be asked during the demonstration.

8 Upload Checklist

Before submitting, ensure that your group has included all of the following files:

- TASLock.java
- Your optimised implementation
- Main.java
- The results of your experiment

9 Submission

Your solution will be submitted via **ClickUp**. **All group members are required to submit.** Your submission should include:

- The source code mentioned in the upload checklist.
- The student numbers of all group members.
- Clear comments identifying the original TAS implementation and the optimised implementation.
- The results of the contention experiment.

All required source files must be placed in a single ZIP archive. The archive must be named according to the following format: **uXXXXXXXX.zip** where **XXXXXXXX** is your student number.

No late submissions will be accepted!

10 Marking Rubric

Assessment Item	Marks
TAS Lock implementation	2
TAS Optimisation	3
Contention experiment and results	2
Conceptual Question 1	3
Conceptual Question 2	3
Total	13